

上位机工具：怎么连、怎么推、怎么发命令

这份只回答三个问题：要做什么 → 敲哪一行 → 看见什么算成功。

原理与判据不在这里（协议在 \$6，工具为什么存在在 \$7，命令的权威表在 report/commands.md）。

运行时只用 Python 3 标准库、Node.js 18+（只用内置 node: 模块）和 Windows 自带的 PowerShell，

三个都不需要装第三方包；ffmpeg 只有推自己的视频时才需要。

0. 三分钟上手（照抄这三条）

步	在哪敲	命令	看见什么算成
1	Windows 终端 (cmd 或 PowerShell)	ping 192.168.1.10	有回包。这一跳同时验了位流——ICMP 应答是 PL 里自己算的，不经过 PS
2	仓库根	python src/host/video_sender.py --demo	先打 [LINK] ping 通...，再打 [TX] -> 192.168.1.10:5001 512x300 RGB565 @...fps pace=... MB/s，屏上换成会动的图卡。双击根目录 send_demo.bat 等价（它 ping 完推 300 s）
3	另开一个终端（串口终端，115200 8N1，COM6）	stat 后回车	回一行 [STAT] ctrl thr=... src=... zoom=... bilin=... zsel=... zman=... pub=... sd=... frames=... playing=... sel=... gm=... mode=... geom=...

第 2、3 步之间没有依赖：先推后连、先连后推都行。同一个 COM6 只容一个写入者——开着串口终端就不能

再用脚本发同一口（PowerShell 会报 UnauthorizedAccessException）。

1. 要装什么、不装什么

需要	用来做	备注
Python 3（标准库）	src/host/video_sender.py、src/host/udp_push.py 两条推流	默认推流走这里，不需要 numpy/requests 之类
Node.js 18+	所有 src/host/*.mjs（推流、读回、命令验收）	实测 v24；只用 node: 内置模块
PowerShell 5+	串口收发（board/*.ps1）	用自带的 SerialPort，本机没有 pyserial
ffmpeg（可选）	把 mp4 解成裸流	只有推自己的视频才需要；发内置测试图不需要

2. 连上板子（一次性）

- 网线接 PL 侧网口（PHY 是 RTL8211，挂在 PL，不是 PS 网口）。
- PC 网卡静态地址 192.168.1.100 / 255.255.255.0；板卡固定 192.168.1.10，视频端口 UDP 5001。
- 板侧下载（JTAG，不写 QSPI）。前提：Vivado 与 Vitis 2025.2.1 的 bin 在 PATH 里，下面用裸命令名。

三条必须按顺序（先起 hw_server，否则第一句就是 Invalid target）：

```
xsdb build\tcl\ps_jtag_boot.tcl
vivado -mode batch -source build\tcl\program_pl.tcl
xsdb build\tcl\ps_app_reload.tcl
```

只看 HDMI 出图、不需要发命令的话，第三步可以省。串口是 COM6 / 115200 / 8N1。

三条跑完后串口应打出 [BOOT] video_pipeline PL-UDP control plane、[CFG] axi_gpio_2 @41220000 ok

与 [CFG] gamma_window @41220000 ok。后两条就是"elf 与 bit 配套"的读数（地址硬编码，

位流里没这个从设备时不会编译失败，只会"写了没反应"）。

3. 推流：按"要做什么"选一条

3.1 只想看画面动 / 拍一段演示

```
python src/host/video_sender.py --demo # ping + 内置测试图, 发满 300 s (send_demo.bat 等价)
python src/host/video_sender.py --demo --seconds 30 # 只发 30 s
```

内置图卡是"渐变底 + 每帧下移的白线 + 横移红块", 用来看**帧在不在动、换帧整不整帧**。

3.2 推自己的视频

```
python src/host/video_sender.py --input 我的视频.mp4 --fps 30 # 需要 ffmpeg
```

ffmpeg 把任意片源解成 PL 画幅 512×300 RGB565 (面板是 1024×600, 输出侧才 ×2): 保比例缩放 + 居中补边,

不拉伸变形。没有 ffmpeg 时 --demo 照发内置图, 而 --input 会明说"没找到 ffmpeg"并退出——

不静默降级、不假装在放。发送端只按帧边界切片, 末尾不足一帧的残片直接丢掉并报数;

半帧会让板端"这一帧少一行"变成常态, frames_bad / rows_miss_max 从此没法对账。

3.3 做有对照条件的实验

```
python3 src/host/udp_push.py --pattern edge --fps 15 --count 600 # 整屏逐帧黑白交替: 判换帧是否原子
python3 src/host/udp_push.py --pattern edge --count 300 --drop-every 500 # 确定性丢包: 板上必须整帧不上屏
node src\host\video_sender.mjs --test frameid --fps 15 --count 200 # 逐帧丢字 / 陈旧帧对照
node src\host\video_sender.mjs --file - < raw.rgb565 # 裸流从 stdin 进
```

src/host/udp_push.py 与 src/host/video_sender.py 协议完全一致; src/host/video_sender.mjs

是同一套 [u32 LE 偏移][载荷] 协议 + 一整套自带对照的图案, 做归因实验用它 (要 Node.js 18+) 。

三条命令的成功读数分别是: video_sender.py 打

[LINK] ping 通... 加 [TX] -> 192.168.1.10:5001 512x300 RGB565 @...fps pace=... MB/s , udp_push.py

打同样的 [TX] 行, video_sender.mjs 打 [TX] 192.168.1.10:5001 512x300 RGB565 @15fps test=... pace=... MB/s src=...。

推完都退出码 0; --count N 到数自己退。

分辨率的说法只有一个: 面板 1024×600, PL 处理画幅 512×300 (输出侧 ×2) , 所以三个发送端发出去的都是

512×300 的帧; --raw 1024x600 这类参数是"源有多大", 不是"板子收多大"。

4. 参数表 (一张看全: 哪条命令有、哪条没有)

参数	默认	哪个工具有	说明
--ip / --port	192.168.1.10 / 5001	三个都有	板卡地址与端口
--src	192.168.1.100	仅 .mjs	本机绑定地址; 传 "" 走默认路由 (src/host/video_sender.mjs:166) 。另两支不绑地址, 走系统默认路由
--fps	15 (.mjs 、udp_push.py) / 30 (video_sender.py)	三个都有	帧率。线速附近请配合下面的匀速
--count N	0 (一直发)	三个都有	发满 N 帧退出
--seconds N	12	仅 video_sender.py	只在带 --demo 时生效 (video_sender.py:209 那一行的判据是 a.demo and ...> a.seconds) ; send_demo.bat 显式传 300, 所以双击是 5 分钟
--no-ping	关	仅 video_sender.py	跳过开推前的那次 ping (板子已确认在跑、或做"应答器变哑"实验时用)

参数	默认	哪个工具有	说明
<code>--demo</code>	关	仅 <code>video_sender.py</code>	开 push 前先 ping 板子，并按 <code>--seconds</code> 到点自己停。它 不是 "只发内置图": <code>--input</code> 片子 <code>--demo</code> 就是"ping + 发这部片子到点停", <code>send_demo.bat</code> 拖文件进来走的就是这一支 (<code>send_demo.bat:30</code>)
<code>--input F / --raw WxH</code>	— / 源尺寸	仅 <code>video_sender.py</code>	任意片源 (mp4/mov/avi 走 ffmpeg; 裸 RGB565 帧流配 <code>--raw</code> 说清源有多大)。 <code>--raw</code> 说的是源，不是板子收多大
<code>--pattern NAME</code>	—	仅 <code>udp_push.py</code>	内置测试图名 (<code>edge</code> 等); <code>.mjs</code> 那边同一件事叫 <code>--test</code>
<code>--file F</code>	—	<code>udp_push.py</code> 、 <code>.mjs</code>	推一份裸 RGB565 帧流，一帧 307200 B; <code>--file -</code> 从 stdin 进
<code>--pace- mbps</code>	15 (<code>.mjs</code> 、 <code>udp_push.py</code>) / 20 (<code>video_sender.py</code>)	三个都有	包内匀速：一整帧 221 包若以线速倾泻会打爆板端入包 FIFO; Node 侧旧拼写 <code>--pace-mpbps</code> 仍接受 (<code>src/host/video_sender.mjs:39</code> 两名都读)
<code>--no-pace</code>	关	仅 <code>.mjs</code>	关掉匀速 (做压力实验时才用)
<code>--mtu- payload</code>	1392	<code>udp_push.py</code> 、 <code>.mjs</code>	必须 8 的倍数，见 §6; <code>video_sender.py</code> 没有这一支开关，固定用默认载荷
<code>--drop- every N</code>	0	<code>udp_push.py</code> 、 <code>.mjs</code>	每 N 包确定性丢一个 (演示坏包恢复，不是随机)
<code>--dump FILE / -- test NAME / --ref</code>	关 / — / —	仅 <code>.mjs</code>	把帧字节落盘供离线核对 / 图案选择 (见下表) / 对照参考帧

`udp_push.py` 的其余参数用 `--help` 看，命名一致但少几项。

`--test` 的图案各管一件事，选错了就看不出问题：

图案	看得出现象	看不出的
<code>bars</code>	横向黑纹 (丢行)、黄块拖影 (重复帧)、左缘奇偶行标记 (行序错乱)	逐帧丢字
<code>grad</code>	量化、色带	时序类
<code>edge</code>	换帧是否原子 (非原子会看到灰行/残影)	空间定位
<code>move</code>	谁在动，一眼分清屏幕上是哪一路源	位级错误
<code>blocks hold wordid</code>	地址映射、字边界对齐	逐帧丢字
<code>frameid</code>	逐帧丢字 / 陈旧帧 (每个字写着来自第几帧)	——

5. 串口命令：怎么用，而不是背

打开串口的两种方式，任选一种 (**同一时刻只能有一种**)：

```
board\uart_cap_once.ps1 -Port COM6 -Seconds 14 -Out cap.txt      # 只收不发，抓一段自发输出
board\uart_cmd_script.ps1 -Port COM6 -File board\cmd_battery_v81.txt -DelayMs 900 -Out cap.txt
```

第二条是"按行发清单"，每行前面 `echo >> <行>`，所以捕获能按命令切片；默认 `-Out` 落到

`board/uart_script_capture.txt` (已被 `.gitignore` 挡住，不入库；随包复核认同行点名的跟踪件)。

要一条一条手敲，就开任意串口终端 (115200 8N1)。

这张表是"想看什么 → 敲什么 → 屏上/串口出什么" (完整动词表与拒收条件在 `report/commands.md`)：

要看	敲	该看见
换片源	<code>src 0 / src 1 / src 2 / src auto</code>	OSD SRC: 那格跟着变 (0=图卡 1=DDR(网络) 2=DDR 并起播 SD)

要看	敲	该看见	
开/关某一级算法	pipe 000010000 然后 pipe show	右窗（或整屏）立刻变； pipe show 念回九位	
调二值化阈值	th 60	二值化那级的黑白分界跟着动	
缩放	zoom 1.5 / zoom auto / zoom fit / zoom off	画面借率变，不重传流	
旋转	rot speed 3 然后 rot auto 1，看状态 rot show	右窗连续转，每帧 3 个度（0.7）	
把缝拖到某处	split 50 / split 20 / split swap 1 / split show	竖缝换位置； split screen 回整屏处理图	
双线性对照	bilin off 再 bilin on	斜线台阶 ↔ 平滑，一眼分别	
逐像素比对/拍屏前关字	osd off（回来 osd on）	五行字消失，画面本身不动	
Gamma	gamma 1.8 / gamma auto 100 300 20 2000 / gamma manual / gamma off / gamma show	灰阶曲线变；表是 PS 算的 256 项	
SD 那一路	sd files / sd file 2 / play / stop / fill / `autoplay 0	1 / `sd remount`	屏上换成卡里的序列
片上温度	temp / temp th 70	[TEMP] degC=...（XADC 实测）； OSD 那一格同源	
全部状态	stat / help / `echo 0	1`	[STAT] 一行 / 命令表 / 心跳开关

三条容易敲错的：

- rot 45 这种“设成某个绝对角度”的写法固件里**不存在**（按键 ±1° 才有角度，rot 只认 auto / speed / show），敲下去得到的是拒绝消息，一个位都不写。
- 老写法（SRC0 TH80 ZOOM1 BILIN1 FRAME12）仍然收；裸五位串只回一句等价的九位、一个位都不写（五位控制字已从 RTL 删净）。
- 命令之间的覆盖关系（哪些组合会静默无效，例如 zoom fit 1 下的 zoom 1.5）在 命令优先级记录，不在上面这张表里。

要自动验收这一层，跑：

```
node src/host/uart_cmd_check.mjs --file board/cmd_battery_v81.txt --port COM6      # 105 条命令逐条对回声
node src/host/uart_cmd_check.mjs --dry      # 只打印期望，不碰串口
node src/host/uart_cmd_check.mjs --self     # 这些期望自己的反例（证明它会判不过）
```

末行 RESULT PASS uart_cmd_check 才算过。跑之前两件事都是踩过坑的：① 板子要在默认态（验收要求“末态 == 初态”，之前手动改过缩放或效果就会判不过——判得对，要先把状态清回去，而不是把条件放宽，清单收尾那几条 zoom 1.0、src auto 就是还状态的）；② COM6 没被别的终端占着，否则 PowerShell 报 UnauthorizedAccessException。-File 那条会重写 board/uart_script_capture.txt，该件不入库（.gitignore 已挡），随包复核只认同行点名的跟踪件。

6. UDP 协议与“为什么载荷是 1392”

```
每包      : [u32 小端 byte_offset][RGB565 载荷 ≤ 1392 B]
一帧      : 512 × 300 × 2 = 307200 B ≈ 221 包
```

- 板端 frame_reasm 按 offset 落位，乱序可以拼对；坏包丢弃、不重传，下一帧自动恢复；
- 收到完整帧后 PL 自动把屏幕交给 ETH，不必先发 src 1。

DDR 打包器一个字是 64bit (4 像素)。载荷取 1396 时包边界落在字中间，同一个字被分两次推送、后一次覆盖前一次，于是每帧留下约每两个包一处的 4 字节洞，洞里是 0x0000，屏上就是均匀散布的黑点。

打包器从 v6.4 起按 16bit lane 驱动 WSTRB，这个坑已经被硬件堵掉（src/rtl/eth/axi_frame_saver64.v:43；同一块板、同一版 bit 复测：1396 时命中率 100.0%、空洞 0）。默认仍建议 1392：少发约 0.3% 的重复 beat，而且"一个包不跨两个 64bit 字"这个性质让包内相位分析还能用。--mtu-payload 只为做这个对照实验而存在。

7. 工具清单与被谁调用

build/gates.sh（发布前检查脚本，逐项打 PASS/FAIL）与 build/board_verify.sh（上板复现脚本）会点名的，才是"必须有"的；其余是量测/复算工具。

脚本	一句话	被谁调用
src/host/udp_push.py	零依赖推流（协议、限速、确定性丢包）；--pattern edge 是"换帧原子性"的现场对照	手工；report/host_guide.md §3.3 的推流对照
build/build_ps_app.py	不用 IDE 把 src/ps 编成 ELF，并做成品自检（入口 = _boot、_vector_table = 第一个 LOAD 的 VirtAddr、基址落在 [0x00100000, 0x3FEF0000] 且不为 0、.text 体积下界、七个符号必须在）；退出码 2/1/3 分别指"输入不在/编译失败/成品不可执行"	report/build.md §1；与 Node 版李生工具（build/ps_app.mjs）产出逐字节相同（md5 57fa442a7eaf...，即板上那一版）
src/host/doc_enc_check.mjs	手写文件（.md .v .c .h .mjs .sh .ps1 .tcl，范围 report/board/src/sim/tools/build/tcl 与 skills/）必须 UTF-8 无 BOM、无 CR 混排、无替换符/私用区	build/gates.sh
src/host/doc_currency_check.mjs	文档里点名的 build/frozen_rNN/ 必须盘上真有、旧编号不许写成"当前默认"	build/gates.sh
src/host/line_cite_check.mjs	交付文档里的 文件.v:NNN 引用：硬错（决定退出码）= 文件不在树里 / 行号越过文件末尾 / 「例化者」列指错 / 逐字抄的固件回声与所引那几行对不上；"锚点不在那几行"（--list-soft）与"这句在该文件里找不到、像转述"（--list-echo）只列清单不判不过。--self 的 15 条对照合厂商豁免 2 条	build/gates.sh 第 20 项（2026-10-02 起进这份检查表；此前只有人工工跑）
src/host/demo_cmds.mjs	演示脚本里的命令块逐条对固件解析器（不碰板子）；--emit 抽排练清单	build/gates.sh
src/host/ps_hb_check.mjs	不碰板子：从固件源码原文抠出"PS 心跳/ps_hold 那一半"必须同时成立的事实逐条钉（17 条），--self 用 10 条变异证明它会判不过	build/gates.sh
src/host/uart_cmd_check.mjs	§5 那条串口回归清单（105 条命令）	build/board_verify.sh
src/host/pipe_len_check.mjs	pipe 这一条命令的"唯一说法"离线核对：位号四份一致、五位一条都不写、退役不退半截（A/B/C/D 四组，不碰板子）	build/gates.sh
src/host/udp_sink_check.mjs	本机 UDP 环回自检：协议、切片、匀速这三件事对不对，不依赖板子	手工（改发送端后必跑）
src/host/health_read.mjs	JTAG 读健康快照 19 条 lane（名单就是 src/host/health_read.mjs:334 那一行 want：lane0~9 加 23~31；越界的号读回 32'hDEAD_BEEF 兜底，src/rtl/top/system_top.v:244）；--json 出机器可读对象，--gapclr 归零帧间隔统计	build/board_verify.sh、board/README.md
src/host/geom_check.mjs	缩放/旋转的几何读数与预期公式对账	build/board_verify.sh、board/README.md
src/host/arb_handover_test.mjs	无人值守的仲裁交接：静默、推流、停、再推四步连着走，100 ms 密度采 lane30，八条 PASS/FAIL（更早那一版是七条）；--selftest 不打板子	build/board_verify.sh
src/host/video_sender.mjs	推流（§3、§4）	build/board_verify.sh（经 arb_handover_test.mjs 起）、演示

脚本	一句话	被谁调用
src/host/make_sd_video.mjs	把 mp4 转成 SD 播放要的裸帧序列 (--in a.mp4 --out E:)	board/README.md
src/host/metrics.mjs	两次 health_read --json 的差值算成抖动/丢包指标; --selftest 验算数本身	手工; 一次实测落盘在 board/evidence_r41/
src/host/ddr_verify.mjs	只回读 DDR 两个 bank 并落盘 data/measured/ddr_dump.out	作者留存, 不随包发布
src/host/ddr_stale.mjs	反解每个 16bit 字来自第几帧, 给出丢字率、游程分布、错帧计数	作者留存, 不随包发布
src/host/ddr_holemap.mjs	把回读结果按行段画空洞分布	作者留存, 不随包发布
src/host/ingress_probe.mjs	定点注入: 只灌 K 个包 (每像素写"字号+一个没用过的帧号") 再回读, 报落位率与"从第几个字开始丢", 直接指出是哪一级缓冲不够	作者留存, 不随包发布
src/host/measure_v63.mjs	一条命令走完"推 frameid → 发完 → 回读两 bank → 相位对照"	作者留存, 不随包发布
src/host/card_preview.mjs	把 sim/tb_v83_card_render.v 倒出的像素转储渲染成 PNG, 图卡观感一分钟能看到, 不用等一轮构建+上板	作者留存, 不随包发布
src/host/interp_study.mjs	动手写硬件之前先算清"插值在本项目的几何区间里值不值" (inv=256/512 两端小数位恒为 0, 双线性逐像素等于最近邻)	作者留存, 不随包发布
src/host/lane30_watch.mjs	高频盯 lane30 那一位 ([秒=45] [间隔ms=150])	作者留存, 不随包发布
src/host/temp_formula_check.mjs	XADC 温度公式与固件换算对照	build/gates.sh
src/host/repo_path.mjs	公共路径/落盘函数, 被上面几个 import	——

8. 故障排查 (现象 → 先查什么 → 用哪条命令确认)

现象	先查	用什么确认
ping 不通	是不是插了 PS 网口; PC IP; 位流是否已下载 (ICMP 在 PL 里)	ping 192.168.1.10 ; 不通就重跑 §2 第 1、2 条
推流无画面	片源归属、eth_live	node src/host/health_read.mjs --json 看 eth_live/owner_eth ; 把推流那支的 --fps 降到 15 (这一支 health_read 没有 --fps) ; 串口 stat 看 src=
屏上均匀黑点	分包长度 (\$6) ; 或 --pace-mbps 太大把入包 FIFO 打满	stat 与 health_read 看 drop_words
画面花/错位	网线; 杀软流量扫描; 网卡双工	强制全双工 1G 后重推; --test bars 看行序
串口无响应	ELF 是否下载; COM 号; 115200 8N1; 有没有别的终端占着端口	重跑 §2 第 3 条; 换一个串口终端
验收脚本卡住	hw_server 是否在跑; xsdb 与 vivado 的调用顺序; 读 DDR 前要不要 rst -processor	按 §2 的三条顺序重来
uart_cmd_check 报"末态≠初态"	板子之前被手动改过状态	先敲 §5 收尾那一串清回默认, 再跑
板子几分钟后不回 ping	已知现象: ICMP 应答器会随运行变哑 (与载荷长度无关)	断电重上; 演示改走 SD + 图卡两路